Metro Ticketing System using ServiceNow - Capstone Project Documentation

1. Introduction

Project Title: Metro Ticketing System using ServiceNow

2. Project Overview

Purpose: To automate metro ticket booking and QR-based ticket validation using ServiceNow platform.
Features: Ticket creation via Service Catalog, automated fare calculation, QR code generation, dashboards, notifications.

The Metro Ticket Booking System is a ServiceNow-based automation solution designed to digitize and streamline the process of metro ticket reservation, fare calculation, and ticket validation using QR technology. The system eliminates manual ticketing processes and reduces passenger wait time by providing a fully automated, self-service booking experience.

The platform is built using the Service Catalog framework, where users can select their journey details such as source station, destination station, travel date, and number of passengers. Based on the selected route, the system dynamically calculates the fare using predefined business rules, ensuring accurate and consistent pricing without manual intervention.

Once the booking is submitted, a ticket record is generated automatically in ServiceNow. The system then triggers backend automation scripts to generate a unique QR code for each ticket, which acts as a digital travel pass for validation during metro entry. This QR code contains essential ticket information such as ticket number, route details, journey date, and passenger count.

To enhance monitoring and operational visibility, the project includes real-time dashboards that display booking statistics, revenue insights, and usage trends. Additionally, automated email notifications are sent to users upon successful booking, providing them with ticket confirmation and QR code access.

Overall, the solution demonstrates how ServiceNow can be used beyond IT service management to build real-world transportation automation systems with workflow orchestration, scripting, and data visualization capabilities.

3. Architecture

Frontend: ServiceNow Service Portal / UI Pages
Backend: ServiceNow Business Rules, Script Includes, Client Scripts
Database: ServiceNow Tables (Custom Ticket Table / RITM), storing ticket and passenger data

The Metro Ticket Booking System is built on a layered ServiceNow architecture that separates user interaction, business logic, and data storage to ensure scalability, maintainability, and automation efficiency.

1. Presentation Layer (Frontend)

The frontend is implemented using ServiceNow Service Portal and UI Pages, which provide the user interface for passengers to book metro tickets. Users interact with dynamic forms where they enter journey details such as source station, destination station, travel date, and number of passengers. Client Scripts are used on the frontend to perform real-time validations and to enhance user experience, such as validating inputs and dynamically triggering fare preview calculations.

2. Application Logic Layer (Backend Processing)

The backend layer is responsible for all business logic and automation. It is implemented using:

Business Rules: Triggered automatically when records are inserted or updated. They handle fare calculation, ticket number generation, and QR code initiation after ticket creation.

Script Includes: Reusable server-side JavaScript modules used for complex logic such as fare computation between stations and QR code generation logic.
Workflow / Flow Designer (if used): Orchestrates the ticket lifecycle from creation to confirmation and notification delivery.

This layer ensures that all business rules are consistently enforced regardless of how the ticket is created.

3. Data Layer (Database)

All ticketing data is stored in ServiceNow Tables, primarily:

A custom metro ticket table or the RITM (Requested Item) table depending on implementation Associated tables for passenger details and transaction records

These tables store structured data including ticket ID, source station, destination station, fare amount, passenger count, journey date, and generated QR code reference.

4. Integration & Output Layer

Once a ticket is successfully created, the system generates a QR code containing encoded ticket details, which is stored in the record and sent to the user via email notification. The Notification Engine of ServiceNow handles automated email delivery.

5. Reporting & Monitoring Layer

Dashboards and reports are built using ServiceNow Reporting tools to provide insights such as:

Total tickets booked
Revenue generated
Most frequently used routes
Daily booking trends

4. Setup Instructions

Prerequisites: ServiceNow Developer Instance
Installation: Configure service catalog items, create ticket table, configure business rules and client scripts.
Step 1: Create Service Catalog Item

- Navigate to **Service Catalog → Maintain Items**
- Create a new catalog item named *Metro Ticket Booking*
- Add required variables such as:
  - Source Station (Reference or Dropdown)
  - Destination Station (Reference or Dropdown)
  - Journey Date (Date Picker)
  - Number of Passengers (Number Field)

Step 2: Create Backend Data Structure

- Navigate to **System Definition → Tables**
- Create a custom table (e.g., u_metro_ticket) OR use **RITM table (sc_req_item)** if extending catalog flow
- Add required fields:
  - Ticket Number
  - Source
  - Destination
  - Fare

- Amount
- Passenger Count
- Journey Date
- QR Code (Image or URL field)

Step 3: Configure Client Scripts

- Navigate to **Catalog Client Scripts**
- Implement scripts for: ○ Field validation (e.g., source ≠ destination) ○ Real-time fare preview (if applicable) ○ Mandatory field enforcement before submission

---

Step 4: Configure Business Rules

- Navigate to **System Definition → Business Rules** • Create rules to trigger **after insert/update** on ticket table
- Implement logic for:
  - Automatic fare calculation based on route
  - Ticket number generation
  - Triggering QR code generation process

Step 5: Implement Script Includes

- Create reusable Script Includes for backend logic such as:
  - Fare calculation engine
  - QR code generation function
  - Utility methods for ticket processing

Step 6: Configure Notifications

- Navigate to **System Notification → Email → Notifications**
- Create notification triggered on ticket creation
- Attach:
  - Ticket details
  - Generated QR code
- Ensure recipients are mapped to the user who booked the ticket

---

Step 7: Configure Reporting and Dashboard •

Navigate to **Reports → Create New Report**

- Build dashboards showing:
  - Total bookings
  - Revenue trends
  - Route-wise analytics
- Add widgets to Service Portal for real-time visibility

Step 8: Testing the System

- Submit a test ticket via Service Portal
- Verify:
  - Record creation in ticket table/RITM
  - Fare calculation accuracy
  - QR code generation
  - Email notification delivery
  - Dashboard data update

Final Note

Once all components are configure

5. Folder / Component Structure

1. Service Catalog Layer

- Contains the **Metro Ticket Booking Catalog Item** •
- Defines user-facing form variables such as:
    - Source Station
    - Destination Station
    - Journey Date
    - Number of Passengers
- Controls the user interaction flow in the Service Portal

---

2. Client-Side Logic

- Implemented using **Client Scripts and UI Policies**
- Responsible for:
    - Real-time form validation (e.g., preventing same source and destination)
    - Dynamic field behavior (mandatory fields, visibility conditions) ○ Basic client-side calculations or UI feedback

---

3. Server-Side Business Logic

- Implemented using **Business Rules and Script Includes**
- Handles core backend operations such as:
    - Fare calculation based on station mapping
    - Ticket number generation
    - QR code generation logic trigger
- Ensures automation is executed consistently after record creation or update

---

4. Data Management Layer

- Uses **ServiceNow Tables**, either:
    - Custom table (e.g., u_metro_ticket)
    - Or Service Catalog tables like sc_req_item
- Stores all transactional data including:
    - Ticket details
    - Passenger information
    - Fare and journey metadata
    - Generated QR code reference

---

5. Reporting & Analytics Layer

- Built using **ServiceNow Reports and Dashboards**
- Provides insights such as:
    - Total tickets booked
    - Revenue generated
    - Most frequently used routes
    - Daily and weekly booking trends
- Supports operational monitoring and decision-making

Key Structural Principle

The system follows a **layered architecture model**:

- UI Layer → Service Catalog + Portal
- Logic Layer → Client Scripts + Business Rules + Script Includes
- Data Layer → ServiceNow Tables
- Insight Layer → Reports & Dashboards

---

6. Running the Application

Access Service Portal URL Submit ticket via
catalog item
Backend processes run automatically via Business Rules

Step 1: Access the Service Portal
Users open the ServiceNow Service Portal using the instance URL and navigate to the **Metro Ticket Booking** catalog item available under the service catalog section.

Step 2: Fill Ticket Booking Form
The user enters the required journey details, including:

- Source Station
- Destination Station
- Journey Date
- Number of Passengers

Client-side validations (implemented using Client Scripts and UI Policies) ensure:

- Mandatory fields are filled
- Source and destination are not the same
- Input data format is correct before submission

Step 3: Submit Ticket Request
Once the form is submitted, a service request is created in ServiceNow:

- A record is generated in the **RITM or custom metro ticket table**
- The request is assigned a unique ticket number automatically

---

Step 4: Backend Processing
After record creation, backend automation is triggered:

- **Business Rules execute automatically** on insert/update
- Fare calculation logic is applied based on selected route
- Ticket metadata is finalized and stored in the database
- Script Includes handle reusable logic such as fare computation and QR code generation

---

Step 5: QR Code Generation
The system generates a unique QR code for the ticket, which contains encoded information such as:

- Ticket ID
- Source and Destination
- Journey Date
- Passenger Count

This QR code is stored in the ticket record and linked to the user's booking.

Step 6: Notification Delivery Once
processing is complete:

- An automated email notification is triggered
- The user receives booking confirmation along with ticket details and QR code

---

Step 7: Monitoring and Verification
Admins or users can track the booking through:

- ServiceNow record view (RITM or ticket table)
- Reports and dashboards showing booking status and analytics

7. API / Automation Logic

The Metro Ticket Booking System primarily uses ServiceNow's internal automation framework. Most of the business logic is implemented using **Business Rules and Script Includes**, ensuring that ticket processing, fare calculation, and workflow execution are handled within the platform.
However, for QR code generation, an external service is integrated to generate machine-readable ticket identifiers.

---

1. Internal Automation (ServiceNow Platform)
The following operations are handled natively within ServiceNow:

- Triggering Business Rules on record creation and update
- Fare calculation based on predefined station-to-station logic
- Ticket number generation and record management
- Workflow execution for ticket processing lifecycle
- Notification triggering after successful ticket creation

These operations ensure that the system remains fully automated without manual intervention.

---

2. External QR Code Generation API
To generate QR codes for metro tickets, an external QR code generation service is integrated within a Business Rule or Script Include.
Working Flow:

- After ticket creation, the system sends encoded ticket data (Ticket ID, Source, Destination, Date, Passenger Count) to the QR API
- The API returns a QR image URL or base64 encoded image
- This QR image is stored in the ServiceNow ticket record
- The QR code is included in the email notification sent to the user

---

3. Automation Trigger Point

- The QR generation process is triggered **after insert Business Rule**
- This ensures the QR code is generated only after a valid ticket is successfully created

---

4. Data Flow Summary
ServiceNow Ticket Record → Business Rule Trigger → Script Include Processing → QR API Call → QR Image Returned → Stored in Ticket Record → Notification Sent

8. Authentication

The Metro Ticket Booking System leverages ServiceNow's built-in **Role-Based Access Control (RBAC)** framework to ensure secure access to application data and functionalities.

---

1. Authentication Mechanism
User authentication is managed by the ServiceNow platform itself. Users must log in using valid ServiceNow credentials before accessing the Service Portal or backend modules. Session management, login validation, and user identity verification are handled by the platform's security layer.

---

2. Role-Based Access Control (RBAC)
Access to different parts of the system is restricted using predefined and custom roles.
Typical roles include:

- **User Role (itil or custom metro_user)** ○
  ○ Can access Service PortalCan create and view their own ticket bookings
- **Admin Role (admin)**
  ○ Full access to configuration, tables, scripts, and system settings
- **Support Role (optional metro_support)** ○ Can view and manage all tickets ○ Can monitor bookings and resolve issues

---

3. Data Security Controls

Security rules are applied at multiple levels:

- **Table-level security (ACLs):** Restrict access to ticket records based on user roles
- **Field-level security:** Sensitive fields (e.g., internal processing data) are hidden from end users
- **Record-level security:** Users can only view tickets they have created
- **Script protection:** Server-side scripts ensure business logic cannot be bypassed from the frontend

---

4. Portal Security

- Service Portal access requires authentication
- Unauthorized users are redirected to login page
- Form submissions are validated server-side to prevent unauthorized data manipulation

---

5. QR Code Access Control

Generated QR codes are tied to authenticated ticket records, ensuring:

- Only valid tickets generate valid QR codes
- QR data cannot be modified by end users after generation

9. User Interface

9. User Interface

The Metro Ticket Booking System provides a user-friendly interface built using **ServiceNow Service Portal**, designed to simplify ticket booking, improve usability, and present real-time booking information.

---

1. Ticket Booking Form (Service Portal)

The primary interface is a structured Service Catalog form where users input journey details.
Key features include:

- Clean and responsive form layout optimized for end-user interaction
- Input fields for Source Station, Destination Station, Journey Date, and Number of Passengers
- Dynamic validations using Client Scripts and UI Policies
- Conditional field behavior (e.g., mandatory validation, error prompts)
- Real-time feedback before submission to prevent incorrect entries

This ensures that users can complete ticket booking in a single streamlined flow.

---

2. Ticket Confirmation Page

After successful submission, users are redirected to a confirmation page displaying:

- Unique Ticket Number
- Journey details (source, destination, date, passengers)
- Calculated fare amount
- Generated QR code for validation

The QR code is prominently displayed and serves as the digital ticket for metro entry verification.

---

3. Dashboard Interface

An analytics dashboard is implemented using ServiceNow Reporting tools to provide operational insights. It includes:

- Total tickets booked over time
- Revenue tracking based on fare data
- Route-wise usage distribution
- Daily and weekly booking trends
- Visual charts and graphs for quick interpretation

This helps administrators monitor system performance and user demand patterns.

---

4. User Experience Design

The interface is designed with the following principles:

- Simplicity: minimal steps required to book a ticket

- Consistency: uniform layout across portal pages
- Responsiveness: compatible with different screen sizes
- Clarity: clear error messages and confirmations

10. Testing

The Metro Ticket Booking System was validated using a structured manual testing approach to ensure correctness of ticket booking flow, fare calculation logic, and QR code generation.

---

1. Test Scope

Testing was performed across the following core modules:

- Service Catalog Ticket Booking Form
- Fare Calculation Logic
- Business Rules Execution
- QR Code Generation
- Notification Delivery
- Dashboard Data Updates

---

2. Test Scenarios and Cases

Test Case 1: Ticket Booking Submission

- **Input:** Valid source, destination, date, and passenger count
- **Expected Output:** Ticket record is created successfully in ServiceNow table/RITM • **Result:** Pass

---

Test Case 2: Input Validation

- **Input:** Same source and destination selected
- **Expected Output:** System should block submission and show validation error • **Result:** Pass

---

Test Case 3: Fare Calculation

- **Input:** Known route with predefined fare mapping
- **Expected Output:** Correct fare should be calculated automatically via Business Rule • **Result:** Pass

---

Test Case 4: Ticket Number Generation

- **Input:** New ticket submission
- **Expected Output:** Unique ticket number generated automatically • **Result:** Pass

---

Test Case 5: QR Code Generation

- **Input:** Successfully created ticket record
- **Expected Output:** QR code generated using Script Include and external API integration • **Result:** Pass

---

Test Case 6: Notification Delivery

- **Input:** Successful ticket creation
- **Expected Output:** Email notification sent with ticket details and QR code
- **Result:** Pass

11. Known Issues

Despite successful implementation of the Metro Ticket Booking System, certain limitations exist due to platform constraints and external dependencies.

---

1. Dependency on External QR Code API
QR code generation relies on an external service. If the API is unavailable or slow, QR generation and ticket confirmation may be delayed or fail. This introduces a dependency outside ServiceNow's control.

---

2. Limited Offline Validation
The system requires active connectivity to ServiceNow for QR verification and ticket validation. Ofline scanning or verification at metro entry points is not supported.

---

3. Static Fare Configuration
Fare calculation is based on predefined logic or mappings. Dynamic pricing based on peak hours, demand, or real-time congestion is not implemented.

---

4. No Real-Time Payment Integration
The system does not include payment gateway integration. Ticket booking assumes successful confirmation without handling actual financial transactions.

---

5. Basic Fraud Protection
QR codes contain encoded ticket data but do not include advanced encryption or blockchain-level validation, making them theoretically vulnerable to duplication if not controlled properly.

---

6. Limited Scalability Testing
The system has not been tested under high-load conditions (e.g., large-scale concurrent ticket bookings), so performance under heavy traffic is not validated.
.

12. Future Enhancements

The Metro Ticket Booking System can be extended further to improve usability, scalability, and real-world deployment readiness.

---

1. Mobile QR Scanner Integration
A dedicated mobile scanning module can be developed for metro entry points. This would allow real-time scanning and validation of QR codes, enabling faster passenger verification and reducing manual ticket checks.

---

2. Payment Gateway Integration
Currently, the system does not handle financial transactions. Integration with payment gateways (such as UPI, credit/debit cards, or wallet services) would enable complete end-to-end ticket purchasing with secure online payments.